Data Handling

Professor: Horacio Larreguy
TA: Eduardo Zago, Ernesto Dávila and Jose Luis Perez

ITAM Applied Research 2,
17/04/2024

General Perspective

Introduction

Exact Matching

Fuzzy Merges: String Distance Approach

Fuzzy Merges: Embeddings Approach

Deduplication and Clustering: Embeddings Approach

Training and Deploying Embedding Models

Geospatial Analysis

Introduction

- We are going to go over different approaches we can take for merging data.
- Particularly, data where each observation is solesly identified by strings.
- Several examples of these datasets: individual level data identified by name (candidate data, bureaucratic data), Google Maps/ARCGIS/OSM data.
- These strings more often than not contain spelling errors and/or are written differently across data sets.

Main Problem

- Computers cannot identify similar/different strings as humans can.
- Remember that everything is processed in a mathematical representation, so at the most simple form: bits
- Take for example the following two, very similar, but different strings (the word is the same, however, one letter is in upper case and the other one lower case):

```
st1 = 'ITAM'
st2 = 'ITAm'
```

Main Problem

- Notice that Python recognizes these two strings as different objects, even though they refer to the same thing.

```
st1 == st2
#### Output: FALSE
```

- Easily solvable:

```
st1.lower() == st2.lower()
 #### Output: True
```

Main Problem

- This issue, and the fact that some datasets have inconsistent string IDs, lead to two main problems:
  1. Merging different data sets.
  2. Duplicate observations found in data.

Figure: Dataset with duplicated (different) strings

NOM_EST	NOM_MUN
Aguascalientes	Pabellon Arteaga
Aguascalientes	Pabellón de Arteaga
Aguascalientes	San José Gracia
Aguascalientes	San Jose de García
Ciudad de México	Benito Juares
Ciudad de México	Benito Juarez
Ciudad de México	Tlalpan
Quintana Roo	Benito Juarez
Quintana Roo	Felipe Carrillo
Quintana Roo	Benito Juarez
Quintana Roo	Felipe Carrillo Puerto
Quintana Roo	Jose María Morelos
Quintana Roo	Morelos

Exact Matching

- Fuzzy merges are like matchmaking based on similarities rather than exact matches.
- They help us bring together data that might not be a perfect match but are close enough.
- Exact matching, on the other hand, is like finding and pairing up identical bits of information.
- These tools will allow us to seamlessly join different data sets and perform regression analysis.

Left Join

- We have done and learnt tons of exact matching (it is an essential part of data analysis).
- Last semester we focused extensively in how to perform exact merges with R, using the left_join() function.
- In Python, as we have seen, the syntaxis is the following:

```
df = df.merge(df_left, on = 'id', how = 'left')
```

String Matching

- Let's look at the most common problematic case: strings across two different datasets are written differently.
- We are going to work with the candidate data for 2018 collected by OPLEs and the electoral institutes for each state.
- Most of these were imputed manually, using pdf documents, panflets, etc. so the municipality variable recorded has some ortographical mistakes.
- We also have the typical omissions of the full name (e.g. Michocán de Ocampo to Michoacán).

String Matching

ESTADO	DISTRITO/MUNICIPIO/CIRC	NOMBRE	GENERO	CARGO BUSCADO
JALISCO	ATOYAC	LUZ ELENA ESTRADA LUNA	M	FUNC MUNIC
JALISCO	ATOYAC	FRANCISCO RODRIGUEZ GUDIÑO	H	FUNC MUNIC
JALISCO	ATOYAC	MARTHA ESTHELA SANCHEZ JIMENEZ	M	FUNC MUNIC
JALISCO	ATOYAC	GABRIEL FLORES URBINA	H	FUNC MUNIC
JALISCO	ATOYAC	MA. MAGDALENA RIVERA BENITEZ	M	FUNC MUNIC
JALISCO	ATOYAC	REYES RUIZ CHAVEZ	H	FUNC MUNIC
JALISCO	ATOYAC	YESICA RODRIGUEZ GONZALEZ	M	FUNC MUNIC
JALISCO	ATOYAC	JESSICA ELIZABETH MATA VERGARA	M	FUNC MUNIC
JALISCO	ATOYAC	JOSE FLORENTINO RAMOS RODRIGUEZ	H	FUNC MUNIC
JALISCO	ATOYAC	DALIA GUADALUPE ANDRES LOPEZ	M	FUNC MUNIC
JALISCO	ATOYAC	REGULO RAMÍREZ VERGARA	H	FUNC MUNIC
JALISCO	ATOYAC	CINTIA LIZBETH VARGAS RODRIGUEZ	M	FUNC MUNIC
JALISCO	ATOYAC	JOSE NORBERTO RODRIGUEZ DE LA CRUZ	H	FUNC MUNIC
JALISCO	ATOYAC	MONICA CORONEL ORTEGA	M	FUNC MUNIC
JALISCO	AUTLAN DE NAVARRO	J. NICOLAS AYALA DEL REAL	H	FUNC MUNIC
JALISCO	AUTLAN DE NAVARRO	VEIRUTH GAMA SORIA	M	FUNC MUNIC
JALISCO	AUTLAN DE NAVARRO	MIGUEL MARDUEÑO IBARRA	H	FUNC MUNIC
JALISCO	AUTLAN DE NAVARRO	MARIA CONCEPCION RAMOS ZUÑIGA	M	FUNC MUNIC
JALISCO	AUTLAN DE NAVARRO	JOSE JUSTINO CARRANZA PUENTE	H	FUNC MUNIC
JALISCO	AUTLAN DE NAVARRO	ESTHELA JUDITH CORONA MARTINEZ	M	FUNC MUNIC
JALISCO	AUTLAN DE NAVARRO	ABEL MONTAÑO VELAZCO	H	FUNC MUNIC
JALISCO	AUTLAN DE NAVARRO	HILDA ROCIO MALDONADO VELAZQUEZ	M	FUNC MUNIC
JALISCO	AUTLAN DE NAVARRO	JOSE MANUEL VILLASEÑOR PRECIADO	H	FUNC MUNIC
JALISCO	AUTLAN DE NAVARRO	SERGIO ATANAHEL RUELAS GONZALEZ	H	FUNC MUNIC
JALISCO	AUTLAN DE NAVARRO	ANA LILIA BRAMBILA RAMIREZ	M	FUNC MUNIC
JALISCO	AUTLAN DE NAVARRO	OSWALDO DESIDERIO RIVERA LLAMAS	H	FUNC MUNIC
JALISCO	AUTLAN DE NAVARRO	MARIA LUISA RODRIGUEZ ROBLES	M	FUNC MUNIC

Figure: Mexican Candidates 2018

String Matching

- In a normal setting, we would just use the merge function on NOM_MUN as our ID variable.
- For this case, since we know that some municipality names are mispelled, we need to follow a procedure to obtain the highest matching rate.
- First, we must start creating a dataset with just the unique set of identifiers/strings, called the *bridge*.
- This dataset, once we finished the merges, will allow us to merge back to the original dataset without using that much memory and RAM.

String Matching

```
bridge = candidatos[['NOM_EST', 'NOM_MUN']].drop_duplicates(['NOM_EST',
                    'NOM_MUN']).reset_index(drop=True)
bridge
```

	NOM_EST	NOM_MUN
0	CAMPECHE	CAMPECHE
1	CAMPECHE	CALKINÍ
2	CAMPECHE	CARMEN
3	CAMPECHE	CHAMPOTÓN
4	CAMPECHE	HECELCHAKÁN
...	...	...
1362	ZACATECAS	VILLANUEVA
1363	ZACATECAS	ZACATECAS
1364	ZACATECAS	EL PLATEADO DE JOAQUÍN AMARO
1365	ZACATECAS	EL SALVADOR
1366	ZACATECAS	NORIA DE ÁNGELES

1367 rows × 2 columns

String Matching

- We can and should always clean up both dataset identifiers to guarantee a higher matching rate.
- There are several things one can do, most depend on the context.
- The most common ones are setting everything to upper case and (in Spanish) remove any accents.

```
from unidecode import unidecode

def remove_accents(text):
    return unidecode(text)

# Assuming 'bridge' and 'mun' are pandas DataFrames with columns 'NOM_EST' and 'NOM_MUN'

bridge['b_EST'] = bridge['NOM_EST'].str.upper().apply(remove_accents)
bridge['b_MUN'] = bridge['NOM_MUN'].str.upper().apply(remove_accents)

# Translate and remove accents from 'NOM_ENT' column in 'mun' DataFrame

mun['b_EST'] = mun['NOM_ENT'].str.upper().apply(remove_accents)
mun['b_MUN'] = mun['NOM_MUN'].str.upper().apply(remove_accents)
```

String Matching

- Once this is done, we exact match on both municipality and state names and observe the matching rate.

```
bridge = bridge.merge(mun, on = ['b_EST', 'b_MUN'], how = 'left')
nan_c = bridge['code_inegi'].isna().sum()

match_rate = (len(bridge) - nan_c)/len(bridge)

print(match_rate)
#### .972
```

- Given the simplicity of the exercise, match rate is really high.
- However, there are contexts where we would like a 100% matching rate.
- Let's try to achieve it using fuzzy matching.

Fuzzy Matching: String Distance Approach

- As mentioned before, fuzzy matching will allow us to match-make based on similarities rather than exact matches.
- Recall the computer only understands numbers, so how can we define similarity in this context?
    1. Words to mathematical representation: one-hot encoded vectors, word embeddings[1]
    2. String distance formulas: Levenshtein Distance and Jaro-Wrinkler.

[1] Which we will cover later.

Levenshtein Distance

- The Levenshtein Distance is an algorithm that calculates the minimum number of edits (insertions, deletions, or substitutions) required to transform one string into the other.
- Example: $Lev(cart, cargo) = 2$ (Why?)
  1. cart $\longrightarrow$ carg (substitution of t for g),
  2. carg $\longrightarrow$ cargo (insertion of o),
- However, most functions in Python return the Ratio, which is defined as:

$$LevRatio = \frac{(a + b - LevenshteinScore)}{(a + b)}$$

- Where a and b are the lengths of the two strings being compared.

FUZZYWUZY

- In Python we have several functions that calculate the distance between two strings. We will focus on the package `fuzzratio`.
- The `fuzzratio` package contains several functions that utilize the Levenshtein distance to generate ratios:

```
from fuzzywuzzy import fuzz

# Standard ratio:
full_r = fuzz.ratio('itam is the best', 'itam is the best') ### 100
full_r_m = fuzz.ratio('itam is the best', 'itam the best') ### 90

# Detects also partial words as similar
full_p_r = fuzz.partial_ratio('itam is the best', 'itam') ### 77

# Order does not mattter, just that strings are the same
full_t_r = fuzz.token_sort_ratio('itam is the best', "the best is itam") ### 100

# Duplicates are considered as one
full_s_r = fuzz.token_set_ratio("itam is best", "itam is is best") ### 100

# Considers upper and lower case letters as the same
full_u_r = fuzz.WRatio("ITAM is best", "itam is best") ### 100
```

FUZZYWUZY

- This package alone could help us merge two different datasets.
- We would have to build several functions and for loops that compare each identifier across datasets.
- Probably overkill, so we will use other packages for that.
- However, this package is really useful to manipulate and generate variables in pandas:

```
df['score'] = df['lastname'].apply(lambda x: fuzz.ratio(x, 'zago') / 100)
```

Fuzzy Merge

- Going back to our candidates dataset, we still need to merge the remaining municipalities:

```
not_matched = bridge[bridge['code_inegi'].isnull()][['NOM_EST', 'NOM_MUN',
                    'b_EST', 'b_MUN']]
```

	NOM_EST	NOM_MUN	b_EST	b_MUN
65	CHIAPAS	METAPA DE DOMÍNGUEZ	CHIAPAS	METAPA DE DOMINGUEZ
116	CHIAPAS	VILLACOMALTITLAN	CHIAPAS	VILLACOMALTITLAN
131	CHIAPAS	CAPITAN LUIS A. VIDAL	CHIAPAS	CAPITAN LUIS A. VIDAL
132	CHIAPAS	RINCÓN CHAMULA	CHIAPAS	RINCON CHAMULA
133	CHIAPAS	VILLAFLORPES	CHIAPAS	VILLAFLORPES
207	COAHUILA DE ZARAGOZA	CUATROCIENEGAS	COAHUILA DE ZARAGOZA	CUATROCIENEGAS
249	CIUDAD DE MEXICO	CIUDAD DE MÉXICO	CIUDAD DE MEXICO	CIUDAD DE MEXICO
274	CIUDAD DE MEXICO	MAGDALENA CONTRERAS	CIUDAD DE MEXICO	MAGDALENA CONTRERAS
275	CIUDAD DE MEXICO	CUAJIMALPA	CIUDAD DE MEXICO	CUAJIMALPA
327	GUANAJUATO	YURIRA	GUANAJUATO	YURIRA
453	MEXICO	ACAMBAY	MEXICO	ACAMBAY
472	MEXICO	COACALCO	MEXICO	COACALCO
474	MEXICO	COCOTILTLAN	MEXICO	COCOTILTLAN
486	MEXICO	ECATEPEC	MEXICO	ECATEPEC
493	MEXICO	IXTPAN DEL ORO	MEXICO	IXTPAN DEL ORO
500	MEXICO	JOCOTITILAN	MEXICO	JOCOTITILAN
501	MEXICO	JOCQUICINGO	MEXICO	JOCQUICINGO
509	MEXICO	NAUCALPAN	MEXICO	NAUCALPAN
556	MEXICO	TLALNEPANTLA	MEXICO	TLALNEPANTLA

Fuzzy Merge: recordlinkage

- To do that, we are going to use the package `recordlinkage`.
- `recordlinkage` is a Python library primarily designed for data matching and deduplication.
- It provides a comprehensive set of tools and algorithms to identify and link records that are the same or related across different data sources.
- Let's install it.

```
!pip install recordlinkage

import recordlinkage
from recordlinkage.index import Block
```

Fuzzy Merge: recordlinkage

- `recordlinkage` is a very flexible package, which will allow us to improve our matching rates and reduce computation time.
- We could try to force a linkage using only b_MUN, comparing every possible combination between the two data sets.
- Let's see how many pairs would that lead us too:

```
indexer = recordlinkage.Index()
indexer.full()

candidates = indexer.index(b_na, mun)
print(len(candidates))
### 91,353
```

Fuzzy Merge: recordlinkage

- The algorithm would have to check 91,353 candidates, which is relatively acceptable.
- However, we can actually just compare municipalities inside a certain state (b_EST), since we know these are written correctly[2].
- This is called *blocking*.

```
indexer = recordlinkage.Index()
indexer.block(left_on='b_EST', right_on='b_EST')
candidates = indexer.index(b_na, mun)

print(len(candidates))
### 2,945
```

[2] Or since these are 32, we could clean that variable really easy.

Fuzzy Merge: recordlinkage

- Now we only have 2,945 possible pairs to compare.
- Blocking is extremely useful to reduce computation time, which goes a long way with bigger datasets.
- It also prevents the algorithm to confuse between places that are called the same, but are located in different states (Benito Juarez is a borough in Mexico City and a municipality in Quintana Roo).
- You can only block on variables which you know are written correctly and are consistent in both datasets.
- Kind of strict matching on b_EST before fuzzy matching on b_MUN.

Fuzzy Merge: recordlinkage

- Given this, let us proceed with the blocking.
- Once we have defined the candidates, we must declare the comparison object, as well as how do we want to compare them.
- The pre-established method is the Levenshtein distance.

```
compare = recordlinkage.Compare()

compare.string('b_MUN', 'b_MUN',
               threshold=0.85, label='MUN')
features = compare.compute(candidates, b_na, mun)
```

Fuzzy Merge: recordlinkage

- The features object contains all available comparisons (using the index to identify them).
- From all possible pairs, let's see how many matched:

```
features.sum(axis=1).value_counts().sort_index(ascending=False)
```

Matched	23
Not Matched	2,922

Fuzzy Merge: recordlinkage

- Now, we must match back with the index of each frame to see which were the municipalities that merged.
- If you are looking for 100% matching rate, you can manually look for the last 14.

```
potential_matches = features[features.sum(axis=1) > 0].reset_index()

mun_lookup = mun[['code_inegi', 'NOM_MUN_INEGI']].reset_index()
mun_lookup['level_1'] = mun.index
na_lookup = b_na[['NOM_MUN']].reset_index()
na_lookup['level_0'] = b_na.index

df_merge = potential_matches.merge(na_lookup, how='left')
final_merge = df_merge.merge(mun_lookup, how='left', on = 'level_1')

final_merge[['NOM_MUN', 'code_inegi', 'NOM_MUN_INEGI']]
```

Fuzzy Merge: recordlinkage

	NOM_MUN	code_inegi	NOM_MUN_INEGI
0	VILLACOMALTITLAN	7071	Villa Comaltitlán
1	VILLAFLORPES	7108	Villaflores
2	CUATROCIENEGAS	5007	Cuatro Ciénegas
3	MAGDALENA CONTRERAS	9008	La Magdalena Contreras
4	YURIRA	11046	Yuriria
5	COCOTILTLAN	15022	Cocotitlán
6	IXTPAN DEL ORO	15041	Ixtapan del Oro
7	JOCOTITILAN	15048	Jocotitlán
8	JOCQUICINGO	15049	Joquicingo
9	TLALTLAYA	15105	Tlatlaya
10	ABASOLO	19001	Abasolo
11	AGUALEGUAS	19002	Agualeguas
12	BUSTAMANTE	19008	Bustamante
13	DOCTOR COSS	19015	Doctor Coss
14	DOCTOR GONZÁLEZ	19016	Doctor González
15	GENERAL TREVIÑO	19023	General Treviño
16	HIGUERAS	19028	Higueras
17	LOS HERRERAS	19027	Los Herreras
18	MELCHOR OCAMPO	19035	Melchor Ocampo
19	RAYONES	19043	Rayones
20	VALLECILLO	19050	Vallecillo
21	GRAL. PLUTARCO ELIAS CALLES	26070	General Plutarco Elías Calles
22	TIXPEUAL	31095	Tixpéhual

Fuzzy Merges: Embeddings Approach

- Recall from the NLP lecture that embeddings are mathematical representations of language.
- BERT embeddings are in reality mathematical representations of **words**.
- This poses a bit of a intuitive problem, since we said that similar words such as *car* and *cat* had very different embeddings representation, while *car* and *bus* had similar ones.
- From now on, we will refer to different types of embeddings, trained for sentence similarity tasks.

Sentence Similarity

- Sentence Similarity is the task of determining how similar two texts are.
- Sentence similarity models convert input texts into embeddings that capture semantic information and calculate how similar they are between them.
- Some models: English, Spanish, Chinese
- To use them, we will not need any heavy coding, since we can use Melissa Dell's package `linktransformer`

Embeddings Approach: linktransformer

- LinkTransformer allows to seemlesly use AI models to standard data frame manipulation tasks like deduplication, clustering, and merges.
- This is important because of three main reasons:
  1. Most of the deep learning models and the way to deploy them are out of reach for regular coders in the social sciences.
  2. Record linkage is one of the most useful skills in these areas.
  3. Using embeddings has proven to be the most efficient and accurate way of performing them.

Fuzzy Merge: linktransformer

- Let's install it and load into the kernel.

```
!pip install linktransformer

import linktransformer as lt
```

- What is impressive about this package is the facility to use Large Semantic Models in two lines of code.
- Let's see how we would merge our last example using a Spanish semantic model:

```
final_merge = lt.merge(b_na, mun, merge_type='1:1',
                       on="b_MUN",
                       model="hiiamsid/sentence_similarity_spanish_es",
                       left_on=None, right_on=None)
```

Fuzzy Merge: linktransformer

```
final_merge = final_merge[final_merge['score']>.85]
final_merge[['NOM_EST', 'NOM_MUN', 'NOM_ENT', 'NOM_MUN_INEGI', 'score']]
```

index	NOM_EST	NOM_MUN	NOM_ENT	NOM_MUN_INEGI	score
2	CHIAPAS	CAPITAN LUIS A. VIDAL	Chiapas	Capitán Luis Ángel Vidal	0.8650339841842651
5	COAHUILA DE ZARAGOZA	CUATROCIENEGAS	Coahuila de Zaragoza	Cuatro Ciénegas	0.8631429672241211
9	GUANAJUATO	YURIRA	Guanajuato	Yuriria	0.9122954607009888
12	MEXICO	COCOTILTLAN	México	Cocotitlán	0.9666234254837036
14	MEXICO	IXTPAN DEL ORO	México	Ixtapan del Oro	0.8659851551055908
15	MEXICO	JOCOTITILAN	México	Jocotitlán	0.8974156975746155
18	MEXICO	TLALNEPANTLA	Morelos	Tlalnepantla	0.9999998807907104
19	MEXICO	TLALTLAYA	México	Tlatlaya	0.9219331741333008
21	NUEVO LEON	ABASOLO	Coahuila de Zaragoza	Abasolo	1.0000005960464478
22	NUEVO LEON	AGUALEGUAS	Nuevo León	Agualeguas	0.9999998807907104
23	NUEVO LEON	BUSTAMANTE	Nuevo León	Bustamante	1.000000238418579
24	NUEVO LEON	DOCTOR COSS	Nuevo León	Doctor Coss	0.9999998807907104
25	NUEVO LEON	DOCTOR GONZÁLEZ	Nuevo León	Doctor González	1.0000001192092896
26	NUEVO LEON	GENERAL TREVIÑO	Nuevo León	General Treviño	1.0000001192092896
27	NUEVO LEON	HIGUERAS	Nuevo León	Higueras	1.0000001192092896
28	NUEVO LEON	LOS HERRERAS	Nuevo León	Los Herreras	0.9999998807907104
29	NUEVO LEON	MELCHOR OCAMPO	México	Melchor Ocampo	0.9999996423721313
30	NUEVO LEON	PARÁS	Nuevo León	Parás	0.9999995827674866
31	NUEVO LEON	RAYONES	Nuevo León	Rayones	1.0
32	NUEVO LEON	VALLECILLO	Nuevo León	Vallecillo	1.0000001192092896
33	QUINTANA ROO	CARRILLO PUERTO	Veracruz de Ignacio de la Llave	Carrillo Puerto	1.0000001192092896
34	SONORA	GRAL. PLUTARCO ELIAS CALLES	Sonora	General Plutarco Elías Calles	0.9170142412185669
36	YUCATAN	TIXPEUAL	Yucatán	Tixpéhual	0.938349723815918

Fuzzy Merge: linktransformer

- We notice that, since we did not cluster, we have some incorrect matches that have a score of 1.
- Luckily, these package also contains a matching with clusters algorithm:

```
final_cluster = lt.merge_blocking(b_na, mun, merge_type='1:1',
                      on="b_MUN", blocking_vars=["b_EST"],
                      model="hiiamsid/sentence_similarity_spanish_es",
                      left_on=None, right_on=None)
```

Fuzzy Merge: linktransformer

```
final_merge = final_merge[final_merge['score']>.85]
final_merge[['NOM_EST', 'NOM_MUN', 'NOM_ENT', 'NOM_MUN_INEGI', 'score']]
```

index	NOM_EST	NOM_MUN	NOM_ENT	NOM_MUN_INEGI	score
2	CHIAPAS	CAPITAN LUIS A. VIDAL	Chiapas	Capitán Luis Ángel Vidal	0.8650339841842651
5	COAHUILA DE ZARAGOZA	CUATROCIENEGAS	Coahuila de Zaragoza	Cuatro Ciénegas	0.8631429672241211
9	GUANAJUATO	YURIRA	Guanajuato	Yuriria	0.9122954607009888
12	MEXICO	COCOTILTLAN	México	Cocotitlán	0.9666234254837036
14	MEXICO	IXTPAN DEL ORO	México	Ixtapan del Oro	0.8659851551055908
15	MEXICO	JOCOTITILAN	México	Jocotitlán	0.8974156975746155
18	MEXICO	TLALNEPANTLA	Morelos	Tlalnepantla	0.9999998807907104
19	MEXICO	TLALTLAYA	México	Tlatlaya	0.9219331741333008
21	NUEVO LEON	ABASOLO	Coahuila de Zaragoza	Abasolo	1.0000005960464478
22	NUEVO LEON	AGUALEGUAS	Nuevo León	Agualeguas	0.9999998807907104
23	NUEVO LEON	BUSTAMANTE	Nuevo León	Bustamante	1.000000238418579
24	NUEVO LEON	DOCTOR COSS	Nuevo León	Doctor Coss	0.9999998807907104
25	NUEVO LEON	DOCTOR GONZÁLEZ	Nuevo León	Doctor González	1.0000001192092896
26	NUEVO LEON	GENERAL TREVIÑO	Nuevo León	General Treviño	1.0000001192092896
27	NUEVO LEON	HIGUERAS	Nuevo León	Higueras	1.0000001192092896
28	NUEVO LEON	LOS HERRERAS	Nuevo León	Los Herreras	0.9999998807907104
29	NUEVO LEON	MELCHOR OCAMPO	México	Melchor Ocampo	0.9999996423721313
30	NUEVO LEON	PARÁS	Nuevo León	Parás	0.9999995827674866
31	NUEVO LEON	RAYONES	Nuevo León	Rayones	1.0
32	NUEVO LEON	VALLECILLO	Nuevo León	Vallecillo	1.0000001192092896
33	QUINTANA ROO	CARRILLO PUERTO	Veracruz de Ignacio de la Llave	Carrillo Puerto	1.0000001192092896
34	SONORA	GRAL. PLUTARCO ELIAS CALLES	Sonora	General Plutarco Elías Calles	0.9170142412185669
36	YUCATAN	TIXPEUAL	Yucatán	Tixpéhual	0.938349723815918

Deduplication/Clustering

- As stated on the first slides, there are two significant issues that arise from inconsistent string IDs.
  1. Merging different data sets.
  2. Duplicate observations found in data.
- We have (partially) solved the first one, which is merging two datasets.
- It might be the case that our dataset contains inconsistent string IDs that identify the same observation over time.
- Like the following example:

Deduplication/Clustering

Figure: Dataset with duplicated (different) strings

NOM_EST	NOM_MUN
Aguascalientes	Pabellon Arteaga
Aguascalientes	Pabellón de Arteaga
Aguascalientes	San José Gracia
Aguascalientes	San Jose de García
Ciudad de México	Benito Juares
Ciudad de México	Benito Juarez
Ciudad de México	Tlalpan
Quintana Roo	Benito Juarez
Quintana Roo	Felipe Carrillo
Quintana Roo	Benito Juarez
Quintana Roo	Felipe Carrillo Puerto
Quintana Roo	Jose María Morelos
Quintana Roo	Morelos

Deduplication/Clustering

- This represent two obvious problems:
  1. For panel data sets: inconsistent regression estimations, as one cannot identify the same unit over time.
  2. For individual level datasets: double counting of the same observation, estimates would be inherently biased.
- Deduplication is like clustering, but we only stay with the first observation from each cluster.
- Luckily, linktransformer provides functions to solve this issue.

Deduplication

- We have seen how to perform standard deduplication in Python:

```
df_dedup = df.drop_duplicates(['NOM_EST', 'NOM_MUN'].reset_index(drop = True)
```

- For the particular case of these municipalities, that will not solve the issue, since they are written incorrectly.
- However, you should first try that.

Deduplication

- As always, before any fuzzy deduplication, we must clean the strings a little bit.
- In these case, again, lower case and remove accents:

```
df['NOM_MUN'] = df['NOM_MUN'].apply(unidecode)

# Perform deduplication:
df_dedup = lt.dedup_rows(df, model = "hiiamsid/sentence_similarity_spanish_es",
                   on=['NOM_MUN'], cluster_type = 'agglomerative',
                   cluster_params = {'threshold': 0.7,
                                     "min cluster size": 2})
df_dedup
```

Deduplication with Blocking

- We managed to remove every duplicated observation.
- Since we did not block on state, we also removed a duplicate that was not the same observation.
- Unfortunately, linktransformer does not provide a function to block and remove duplicates.
- However, we can do it with a for loop:

```
list_states = set(df['NOM_EST'].tolist())
df_final = pd.DataFrame()
for i in list_states:
    df_f = df[df['NOM_EST'] == i]
    df_dedup = lt.dedup_rows(df_f, model = "hiiamsid/sentence_similarity_spanish_es",
                             on=['NOM_MUN'], cluster_type = 'agglomerative',
                             cluster_params = {'threshold': 0.7,
                                     "min cluster size": 2})
    df_final = pd.concat([df_final, df_dedup]).reset_index(drop=True)
df_final
```

Deduplication with Blocking

	NOM_EST	NOM_MUN
0	Aguascalientes	Pabellon Arteaga
2	Aguascalientes	San Jose Gracia
4	Ciudad de México	Benito Juares
6	Ciudad de México	Tlalpan
8	Quintana Roo	Felipe Carrillo
11	Quintana Roo	Jose Maria Morelos
12	Quintana Roo	Morelos

Figure: No blocking

	NOM_EST	NOM_MUN
0	Aguascalientes	Pabellon Arteaga
1	Aguascalientes	San Jose Gracia
2	Ciudad de México	Benito Juares
3	Ciudad de México	Tlalpan
4	Quintana Roo	Benito Juarez
5	Quintana Roo	Felipe Carrillo
6	Quintana Roo	Jose Maria Morelos
7	Quintana Roo	Morelos

Figure: Blocking

Clustering

- Next, we will perform clustering on a candidate dataset from Tanzania.

Figure: Tanzania Candidate Dataset

region	year	
Arusha	1970	Patrick Silverus Qorro
Arusha	1975	Patrick Silvesius Qorro
Arusha	1980	Patrick Syleverius Qorro
Arusha	1970	Damian Buha Geay
Arusha	1975	Damiano Buha Geay
Arusha	1980	Damian Buha Geay
Arusha	1975	Solomon Alexander Ole Saibull
Arusha	1980	Solomon Alexanda Ole Saibul
Arusha	1970	Ahmed M. Kimosa
Arusha	1975	Abdallah Kimosa
Arusha	1980	Kimosa Abdallah Mohamed
Arusha	1970	Joel Salomon Kivuyo
Arusha	1980	Kivuyo Joel Solomoni
Arusha	1965	J. N. A. Kirilo
Arusha	1970	Japhet Nguya Kirilo
Arusha	1970	Merrus R. Merrus
Arusha	1975	Merus R. Merus
Arusha	1980	Merus Richard Merus
Arusha	1965	T. S. Silvine
Arusha	1970	S. S. Silvini
Arusha	1975	Obed Simeon Ole-Mejooli
Arusha	1980	Ole Mojoel Obedi Simeon

Clustering

- From now on, we will just perform these fuzzy operations doing blocking.
- Therefore, for this dataset, we will cluster the names blocking on the region.
- We know that the same individual might be repeated across the same region, but not in a different one.

```
list_regions = set(df['region'].tolist())
df_final = pd.DataFrame()
for i in list_regions:
    df_f = df[df['region'] == i]
    df_dedup = lt.cluster_rows(df_f, model = "sentence-transformers/multi-qa-mpnet-base-dot-v1",
                        on=['name'], cluster_type = 'agglomerative',
                        cluster_params = {'threshold': 0.8,
                                          "min cluster size": 2})
    df_final = pd.concat([df_final, df_dedup]).reset_index(drop=True)
```

Clustering

- We generate the cluster by region variable, which will help us identify each cluster.
- Notice that this step is necessary, since the code is processing and assigning numbers to each region.

```
df_final['cluster_region'] = df_final['region'] + '_' + df_final['cluster'].astype(str)
df_final = df_final.sort_values(by='cluster_region')
df_final
```

- We get the following dataset:

Clustering

Figure: Tanzania Clustered Dataset

index	region	year	name	cluster	cluster_region
327	Arusha	1965	E. M. Sokoine	0	Arusha_0
330	Arusha	1980	Edward Moringe Sokoine	0	Arusha_0
329	Arusha	1975	Edward Moringe Sokome	0	Arusha_0
328	Arusha	1970	Edward Sokoine	0	Arusha_0
310	Arusha	1970	S. S. Silvini	1	Arusha_1
309	Arusha	1965	T. S. Silvine	1	Arusha_1
298	Arusha	1980	Solomon Alexanda Ole Saibul	10	Arusha_10
297	Arusha	1975	Solomon Alexander Ole Saibull	10	Arusha_10
291	Arusha	1970	Patrick Silverus Qorro	11	Arusha_11
293	Arusha	1980	Patrick Syleverius Qorro	11	Arusha_11
292	Arusha	1975	Patrick Silvesius Qorro	11	Arusha_11
320	Arusha	1970	Obed S. Mejool	12	Arusha_12
325	Arusha	1970	Peter Mark Bura	13	Arusha_13
296	Arusha	1980	Damian Buha Geay	14	Arusha_14
295	Arusha	1975	Damiano Buha Geay	14	Arusha_14
294	Arusha	1970	Damian Buha Geay	14	Arusha_14
313	Arusha	1975	Ahmed Amri Dodo	15	Arusha_15
314	Arusha	1980	Dodo Ahmed Amri	15	Arusha_15
323	Arusha	1965	S. Larusai	16	Arusha_16
324	Arusha	1975	S. Larusal	16	Arusha_16
326	Arusha	1965	D. B. Geay	17	Arusha_17
315	Arusha	1965	A. B. Dodo	18	Arusha_18
316	Arusha	1970	A. A. Dodo	18	Arusha_18

Training

- Finally, if we wish to improve our models, and we have a training dataset related to our context, we can easily train and deploy models using linktransformer.

name1	name2	label
Joel Salomon Kivuyo	Joel Solomon Kuvuyo	1
Obed S. Mejool	Obed Ole Majoli	1
M. K. Jumbe	Mohamed Kihago Jumbe	1
P. I. C. Mangatinga	Paul Temoteo Mangatinda	1
T. K. Binafsi	Saidi Tilia Kivuyo	1
...	...	...
B. M. Kavishe	Saidi Tambwe	0
G. J. S. Kimaro	E. M. Makomba	0
C. J. Regia	Yohana Machunga	0
F. M. Subira	T. M. Ndazi	0
Patrick O. Yamo	J. D. Ndomba	0

Training

- Training a model is as easy as the next few lines:

```
    saved_model_path = lt.train_model(
        model_path="sentence-transformers/multi-qa-mpnet-base-dot-v1",
        data=training,
        left_col_names=["name1"],
        right_col_names=["name2"],
        label_col_name="label",
        left_id_name=['id_1'],
        right_id_name=['id_2'],
        log_wandb=False,
        training_args={"num_epochs": 5,
                       "test_at_end": True,
                       "save_val_test_pickles": True,
                       "model_save_name": "multi-mpnet-final", # name of the saved model
                       "opt_model_description": "test",
                       "opt_model_lang":"en",
                       "val_perc":0.2,
                       "large_val": True}
)
```

Deployment

- Once you have finished training your own model, deploying is as easy as using the HuggingFace models available.
- Save the output model in a folder that is easy accessible, and call the same functions as before:

```
df_dedup = lt.cluster_rows(df_f,
                    model = "drive/MyDrive/tanzania_link/models/deduplication/mpnet_cluster_final",
                    on=['name'], cluster_type = 'agglomerative',
                    cluster_params = {'threshold': 0.8, "min cluster size": 2})
```

Geospatial Analysis

- A lot of the available data for research is spatial data, meaning there is an specific location on Earth assigned to each observation.
- Therefore, knowing how to handle this data is incredibly useful when conducting economic research.
- We will take a look at a brief intro to geospatial analysis using geopandas.

```
!pip install geopandas
import geopandas as gpd
```

Spatial Analysis

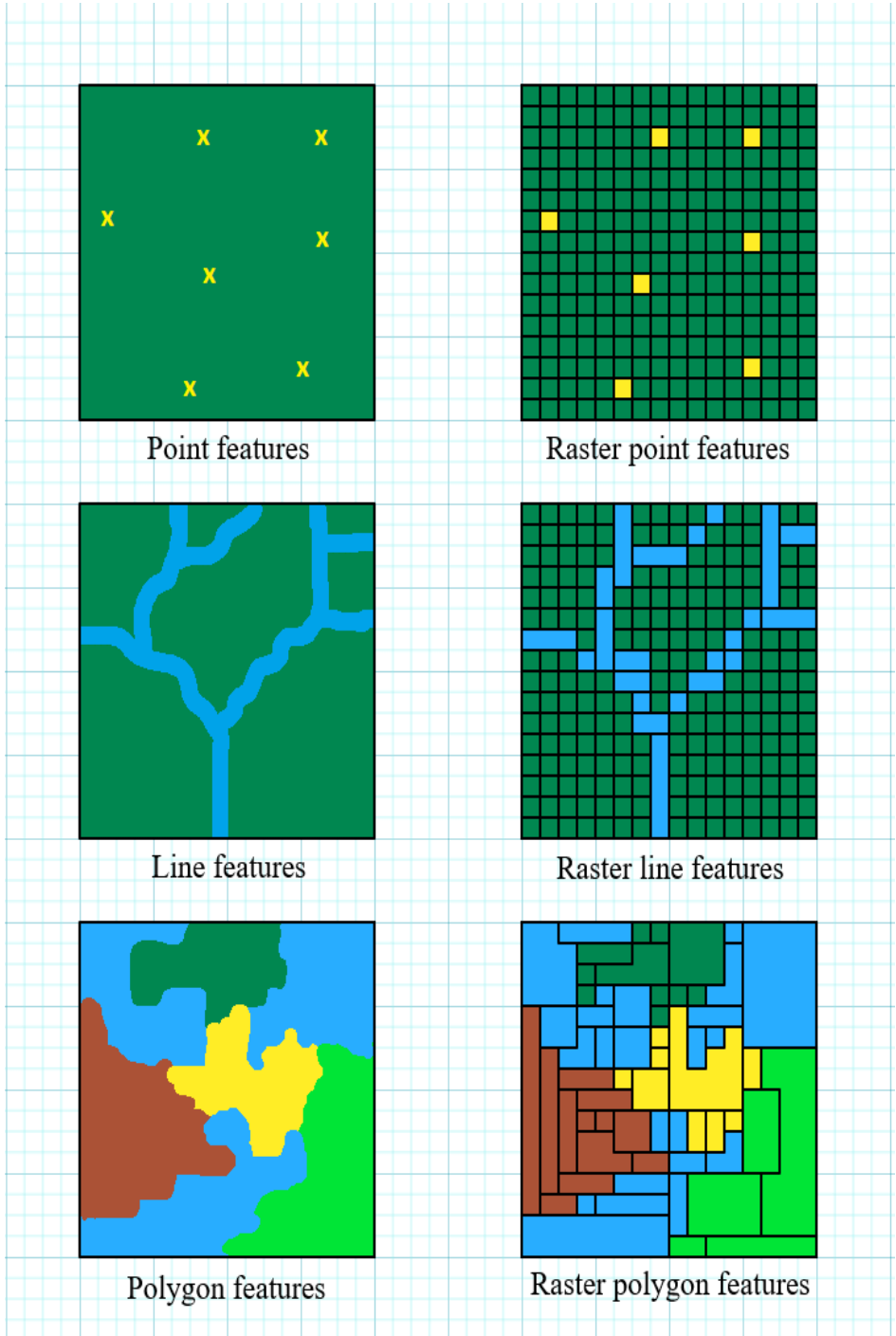
- geopandas is the continuation of pandas but for spatial analysis.
- The package will allow us to perform geospatial operations while maintaining almost the same syntax as in pandas.
- Syntax will be similar, however, spatial data has different datatypes and variables, which we need to understand.
    1. Vector data: describes locations on Earth as discrete geometries.
    2. Raster data: encodes the locations as a continuous surface, basically a grid, which is similar to pixel data

Vector Data

- Vector data describes the locations as discrete geometries, so we will talk about points, lines and polygons.
- A point is an X, Y coordinates location, such as a prison, or an EcoBici station.
- A line is a series of connected points describing a road.
- A polygon will be an enclosure, like an area, which is formed by a closed line.
- This type of data is more common for specific locations (points), and countries, counties or municipalities boundaries (polygons)

Raster Data

- Raster data represents spatial information through a grid of cells or pixels.
- Each cell or pixel contains a value representing a characteristic.
- Common formats include JPEG, PNG, TIFF, and GeoTIFF.
- Raster datasets are suitable for representing continuous phenomena like elevation, temperature, and precipitation.


Point features
Raster point features
Line features
Raster line features
Polygon features
Raster polygon features

Spatial Analysis: geopandas

- geopandas only works with vector data, but it can pair really we with other packages that handle raster data.
- However, for this introduction we will only use vector data.
- In particular we will use data from Mexico City's boroughs and EcoBici locations.
- To start coding we must first know what kinds of files we are going to deal with.
- When working with vector data, we will more often than not encounter **shapefiles** and **geoJSONs**.

Spatial Analysis: geopandas

- Shapefiles are the most common data files.
- They contain three files usually contained inside a compressed folder[3].
  1. The .shp file contains shape geometry.
  2. The .dbf file holds attributes for each geometry,
  3. The .shx file or shape index file helps link the attributes to the shapes.

```
shape = gpd.read_file('../data/poligonos_alcaldias_cdmx/poligonos_alcaldias_cdmx.shp')
```

---

[3]You should never delete any of the files inside the folder.

Shapefile

	CVEGEO	CVE_ENT	CVE_MUN	NOMGEO	geometry
0	09002	09	002	Azcapotzalco	POLYGON ((-99.18231 19.50748, -99.18229 19.507...
1	09003	09	003	Coyoacán	POLYGON ((-99.13427 19.35654, -99.13397 19.356...
2	09004	09	004	Cuajimalpa de Morelos	POLYGON ((-99.25738 19.40112, -99.25698 19.400...
3	09005	09	005	Gustavo A. Madero	POLYGON ((-99.11124 19.56150, -99.11485 19.557...
4	09006	09	006	Iztacalco	POLYGON ((-99.05751 19.40673, -99.05753 19.406...
5	09007	09	007	Iztapalapa	POLYGON ((-99.01692 19.38187, -99.01652 19.381...
6	09008	09	008	La Magdalena Contreras	POLYGON ((-99.20819 19.33674, -99.20859 19.336...
7	09009	09	009	Milpa Alta	POLYGON ((-98.99718 19.22747, -98.99723 19.227...
8	09010	09	010	Álvaro Obregón	POLYGON ((-99.18906 19.39559, -99.18871 19.394...
9	09011	09	011	Tláhuac	POLYGON ((-98.97881 19.32392, -98.97856 19.323...
10	09012	09	012	Tlalpan	POLYGON ((-99.19671 19.30240, -99.19629 19.302...
11	09013	09	013	Xochimilco	POLYGON ((-99.09880 19.32045, -99.09870 19.319...
12	09014	09	014	Benito Juárez	POLYGON ((-99.14762 19.40401, -99.14681 19.403...
13	09015	09	015	Cuauhtémoc	POLYGON ((-99.12951 19.46265, -99.12919 19.462...
14	09016	09	016	Miguel Hidalgo	POLYGON ((-99.19045 19.47044, -99.19058 19.467...
15	09017	09	017	Venustiano Carranza	POLYGON ((-99.10946 19.45292, -99.10895 19.452...

Projections

- CRS information is really important when conducting spatial analysis since it tells GeoPandas where the coordinates are located on Earth.
- If you need to combine two spatial datasets, you need to make sure they are expressed in the same CRS.
- If you need to conduct operations in meters or kilometers you need to obtain the projected CRS for that particular zone of Earth.
- Standard ones for Mexico are: EPSG 6362 (ITRF92) and 6372 (ITRF2008).

```
crs_later = shape.crs
shape.to_crs(epsg=2163, inplace=True)
```

Spatial Analysis: geopandas

- We can now easily get the area of each of the polygons:

```
shape['area'] = shape.area / 1000000
```

- Get the centroid of the polygon, which is a point.

```
shape['centroid'] = shape.centroid
```

- And the boundary:

```
shape['boundary'] = shape.boundary
```

Spatial Relations

- One important feature of spatial analysis is how different geometries and datasets relate spatially to one another.
- One can perform normal attribute joins, as we have seen before.
- Now, the interesting part is going to be how to perform spatial joins.
- For example, let's say we want to obtain the INEGI code for which each EcoBici is located.

EcoBici

	sistema	num_cicloestacion	nombre	calle_principal	calle_secundaria	colonia	alcaldia	latitud	longitud	tipo_ce	candados
0	ECOBICI	1	Río Sena - Río Balsas	Río Sena	Río Balsas	Cuauhtémoc	CUAUHTEMOC	19.433590	-99.167819	4G	27
1	ECOBICI	2	Río Guadalquivir - Río Nazas	Río Guadalquivir	Río Nazas	Cuauhtémoc	CUAUHTEMOC	19.430510	-99.171201	3G	21
2	ECOBICI	3	Reforma - Insurgentes	Reforma	Insurgentes	Tabacalera	CUAUHTEMOC	19.431630	-99.158547	4G	36
3	ECOBICI	4	Río Nilo - Río Pánuco	Río Nilo	Río Pánuco	Cuauhtémoc	CUAUHTEMOC	19.428491	-99.171693	3G	15
4	ECOBICI	5	Río Pánuco - Río Tíber	Río Pánuco	Río Tíber	Cuauhtémoc	CUAUHTEMOC	19.429804	-99.169451	3G	12
...	...	...	...	...	...	...	...	...	...	...	...
475	ECOBICI	476	Lago Como - Laguna de Mayrán	Lago Como	Laguna de Mayrán	Granada	MIGUEL HIDALGO	19.442127	-99.184433	4G	36
476	ECOBICI	477	Lago Iseo - Laguna de Mayrán	Lago Iseo	Laguna de Mayrán	Anáhuac I Sección	MIGUEL HIDALGO	19.440905	-99.181743	3G	24
477	ECOBICI	478	Laguna de Mayrán - Lago Chalco	Laguna de Mayrán	Lago Chalco	Anáhuac I Sección	MIGUEL HIDALGO	19.440818	-99.176961	3G	27
478	ECOBICI	479	Lago Muritz - Av. Marina Nacional	Lago Muritz	Av. Marina Nacional	Mariano Escobedo	MIGUEL HIDALGO	19.444433	-99.179664	Multimedial	24
479	ECOBICI	480	Lago Iseo - Av. Marina Nacional	Lago Iseo	Av. Marina Nacional	Mariano Escobedo	MIGUEL HIDALGO	19.446073	-99.181654	3G	21

480 rows × 11 columns

Spatial Relations

- We have to set up both data frame to the same CRS.
- Let's use the one we started with and perform the merge[4]

```
geometry = [Point(lon, lat) for lon, lat in zip(df['longitud'], df['latitud'])]

# Create a GeoDataFrame
gdf = gpd.GeoDataFrame(df, geometry=geometry, crs=crs_later)
shape = shape.to_crs(crs_later)

# Perform Left Join operation:
gdf = gpd.sjoin(gdf, shape, how='left', op='within')
```

[4] If you encounter an error, pip install rtree before performing any spatial relation operation.

Spatial Relations

- Merging spatial datasets is simple, but there some considerations to be made.
- In the last slide we performed a left join, where we asked geopandas to find the boroughs where each data point was located **within**.
- However, we could also want to see if a line **intersects** a certain set of polygons.
- For that we would use:

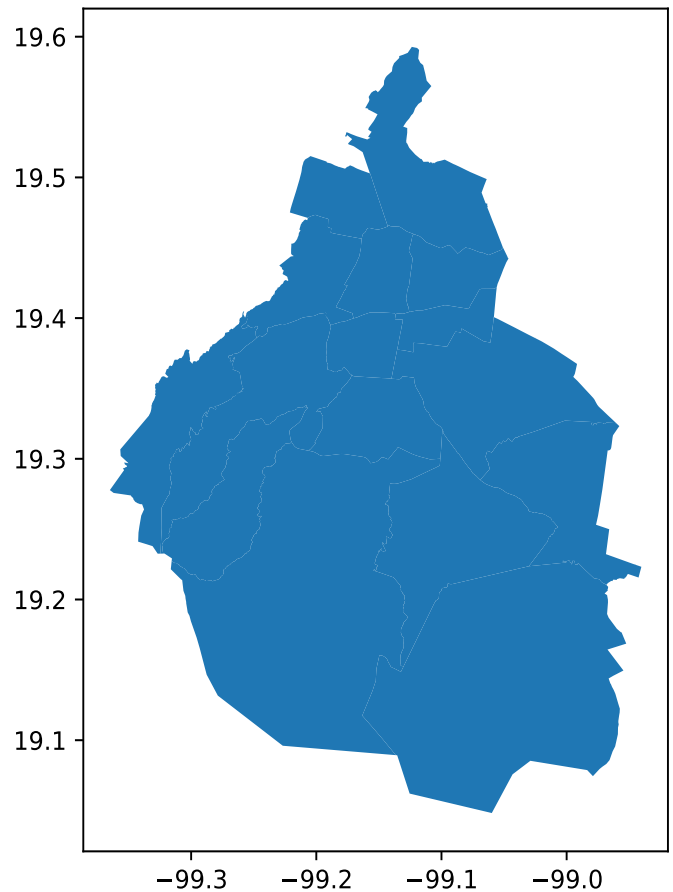
```
lanes = gpd.sjoin(gdf, bike_lane, how='inner', predicate='intersects')
```

Spatial Analysis: Maps

- Probably the most important aspect of spatial analysis is to visualize you spatial relations on a map.
- To do this, the package itself provides great functions and features that will allow us to create nice looking maps.
- Let us start with a simple one:

```
shape.plot(figsize=(10,6))
```

Mexico City Map



Spatial Analysis: Maps

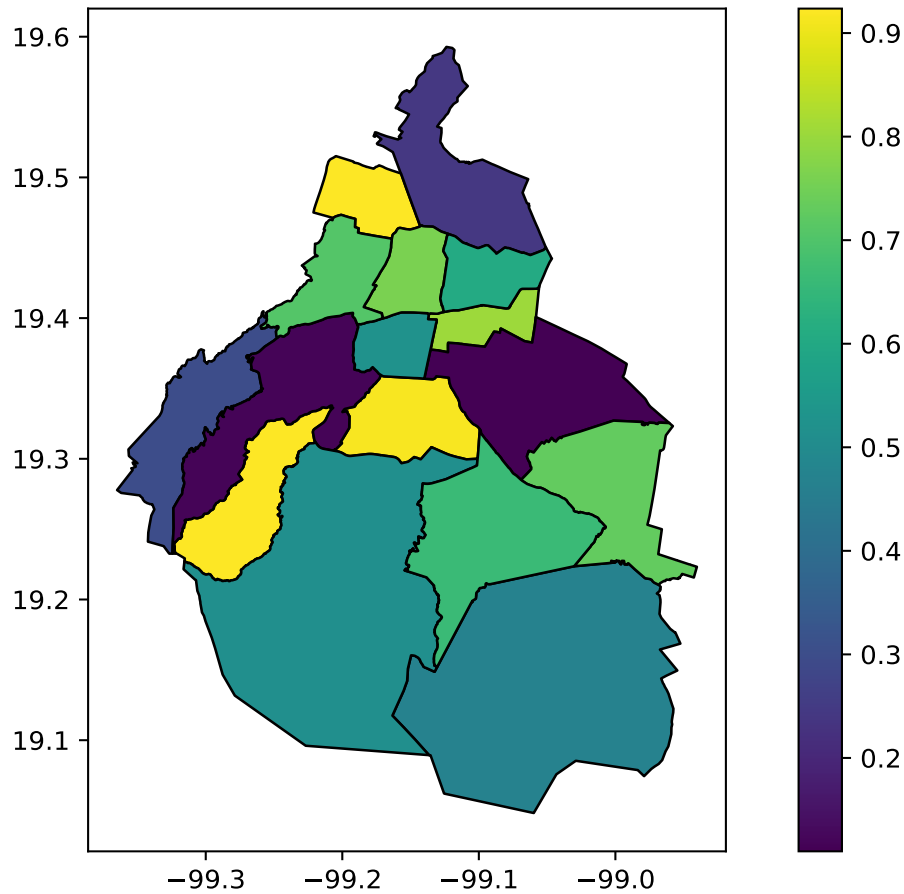
- We can modify them to look better really easy[5]
- Let's say we would like to map a variable that varies across the boroughs:

```
import numpy as np
shape['random'] = np.random.rand(len(shape))

shape.plot(column='random', figsize=(10,6), edgecolor='black',
       legend=True)
plt.savefig('../data/municipalities_map2.pdf', format='pdf')
```

[5] As always, most of the modifications to graphs and maps are on ChatGPT or Stack.

Mexico City Map

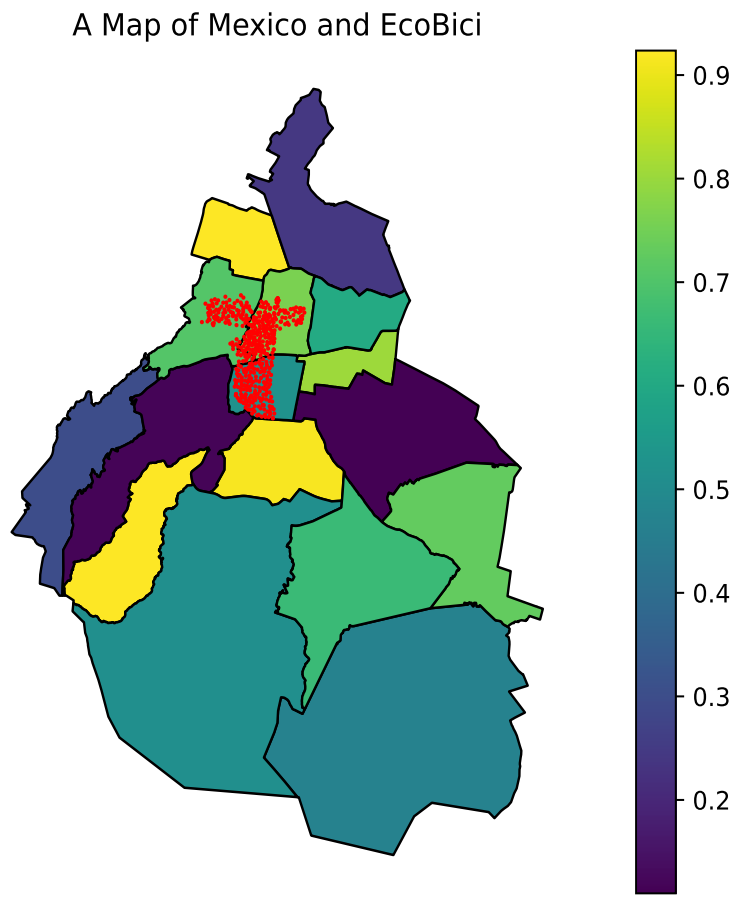


Spatial Analysis: Maps

- We can also plot different spatial datasets on top of each other.
- In this case, EcoBici's frame with points on top of the boroughs polygons.

```
ax = shape.plot(column='random', figsize=(10,6), edgecolor='black',
       legend=True)
gdf.plot(ax=ax, color="red", markersize = .5)
plt.title('A Map of Mexico and EcoBici')
plt.axis('off')
plt.savefig('../data/municipalities_map3.pdf', format='pdf')
```

Mexico City Map



Geolocation: Nominatim (OSM)

- Finally, if we had a set of string locations that we wanted to geolocate (obtain lat and lon.) we could use the OpenStreetMap APIs.
- To use it, we must start by setting up the environment:

```
from geopy.geocoders import Nominatim
from random import randint
import logging
from geopy.geocoders import Nominatim
from geopy.exc import GeocoderTimedOut, GeocoderServiceError

# Defining the key that identifies us as a user:
user_agent = 'user_me_{}'.format(randint(10000,99999))
geolocator = Nominatim(user_agent=user_agent)
```

Geolocation: Nominatim (OSM)

- Once this is done we could start geo-locating places:

```
location = geolocator.geocode('Palacio de Bellas Artes')
location.latitude # 19.435674
location.longitude # -99.141209
```

- More importantly, we could bulk geo-locate a set of strings using a for loop:

```
locs = ['Palacio de Bellas Artes', 'Castillo de Chapultepec']

for i in locs:
    location = geolocator.geocode(i)
    print(location.latitude, location.longitude)
    sleep(1.2)

# 19.435674249999998 -99.14120885319642
# 19.420463249999997 -99.18208657199676
```

QUESTIONS